VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Electronics Engineering (SENSE)

J COMPONENT – REPORT

COURSE CODE / TITLE	BECE320E – EMBEDDED C PROGRAMMING
PROGRAM / YEAR/ SEM	B.Tech III Year/ WINTER 2025-2026
LAST DATE FOR REPORT SUBMISSION	14.04.2026, 11:59 PM
DATE OF SUBMISSION	14.04.2026

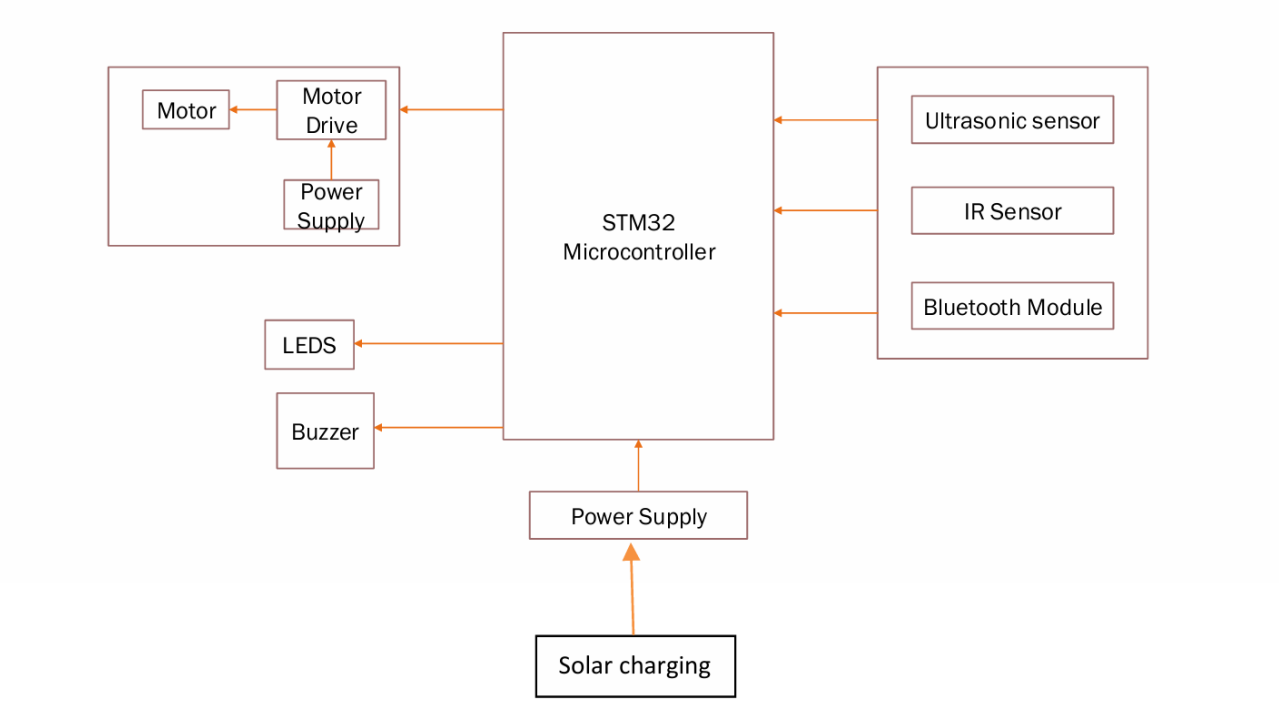
TEAM MEMBERS DETAILS	**REGISTER NO.**	**NAME**
	23BEC1064	HARIPRASATH M
	23BEC1181	MUKESH N
	23BEC1220	DEEPAK A
	23BEC1346	NITHIN V

PROJECT TITLE	Implementation of an Embedded Automated Driving Platform with Battery support		
COURSE HANDLER'S NAME	**Dr.S.Muthulakshmi**	**REMARKS**	
COURSE HANDLER'S SIGN			

OBJECTIVE:

To design and implement a low-cost embedded automated driving platform using an STM32 microcontroller that enables manual, assisted, and fully autonomous navigation in structured indoor environments, incorporating obstacle detection and a solar-based battery support system for enhanced sustainability and reliability.

BLOCK DIAGRAM:



COMPONENTS/ SOFTWARE REQUIRED:

Hardware components		
Components	**Description**	**Quantity**
Microcontroller	STM32F103C8T6	1
Motor driver	L298N	1
DC Motor	-	4
Ultrasonic Sensor	HC-SR04	1
Infrared Sensor	-	2
Bluetooth Module	HC-05	1
Buzzer	-	1
LED	-	2
Battery	18650 Li-ion	2
Solar cell	-	1
Charging module	TP4056	1

DC-DC Buck converter	LM2596	1
Voltage regulator	AMS1117	1

Software Tools
Arduino IDE (with STM libraries)
STM32 ST-LINK Utility
STM32 Cube IDE

PROJECT DESCRIPTION:

The project is dedicated to the development and creation of an embedded automated driving system with built-in battery support, and aimed at low-speed autonomous navigation in controlled situations in an indoor environment. The system is designed based on an STM32 microcontroller as the central processing unit and allows the system to coordinate the actions of sensing, control, and actuation modules effectively.

The platform will have the ultrasonic and infrared sensors to identify the obstacles and help navigate in real-time. Depending on the sensor values, the microcontroller then calculates the environmental values and sends the corresponding signals to the motor driver to move the motor accurately, such as in forward, turning, and stopping. The system also enables versatility in operation with various modes of operation such as manual control through the Bluetooth communication, assisted driving, and complete autonomous obstacle avoidance.

One of the improvements made in this design is that it incorporates a battery-powered power system and a solar-assisted charging system. The main power source is a rechargeable battery, and a solar panel, which is associated with TP4056 charging module, allows capturing the energy all the time and maintaining a safe battery operation. This is added to enhance the energy efficiency of the system and decrease reliance on traditional charging, which makes the platform more sustainable and viable to be implemented in the long term. The entire architecture is planned to be small, affordable and scalable, which can be easily adjusted to applications like smart logistics, contactless delivery systems as well as indoor automation.

CONCEPT LEARNED:

- Understanding of embedded system design using STM32 microcontrollers, including GPIO interfacing and real-time control.
- Implementation of sensor integration techniques, specifically ultrasonic and infrared sensors for obstacle detection and environment sensing.
- Application of motor control mechanisms using PWM signals and H-bridge motor drivers for precise movement and direction control.
- Development of multi-mode operation systems, including manual (Bluetooth-based), assisted, and fully autonomous navigation.

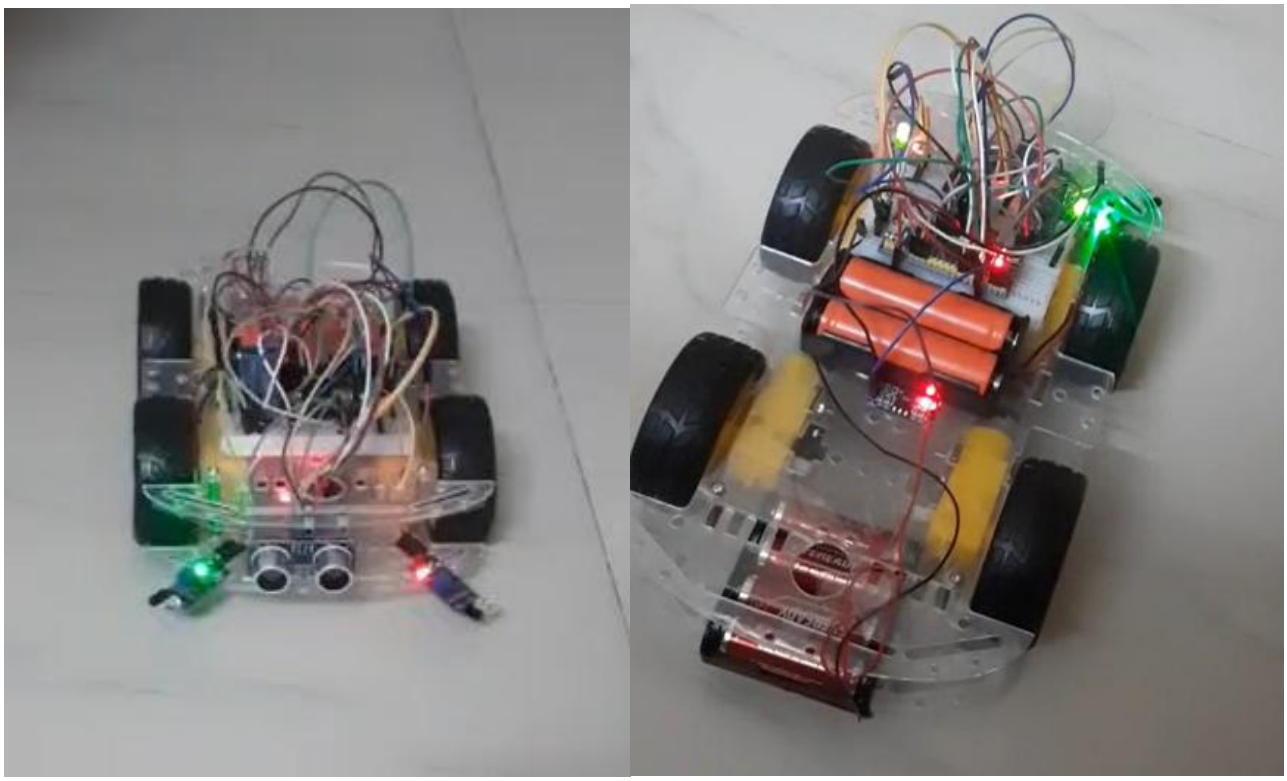
- Knowledge of serial communication protocols (UART) for wireless control using Bluetooth modules.
- Design and integration of a battery management system using the TP4056 module for safe charging and discharging.
- Understanding of solar energy harvesting, including the use of solar panels for supplementary power and improving energy efficiency.
- Exposure to power management strategies for embedded systems to ensure stable and continuous operation.
- Implementation of real-time decision-making algorithms for obstacle avoidance and navigation.

IMPLEMENTATION:

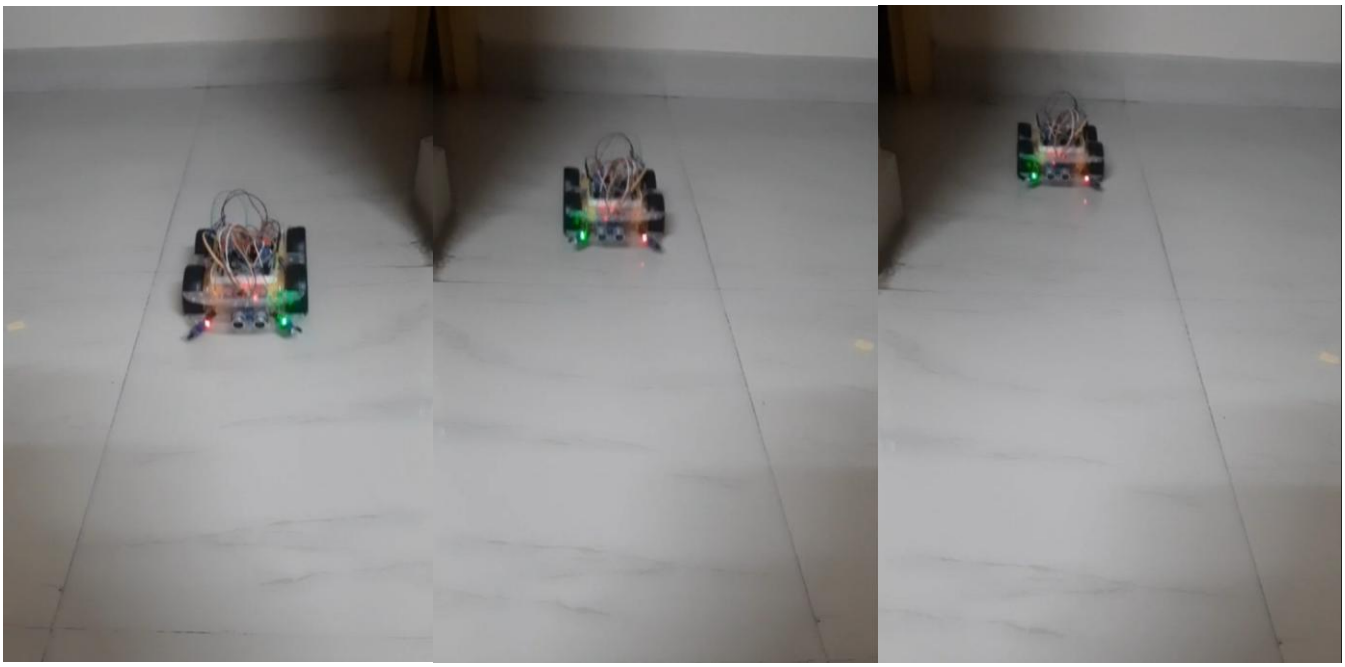
Code algorithm -

1. Main control loop: continuously checks for two primary inputs - data from the HC-05 Bluetooth module and distance measurements from the HC-SR04 ultrasonic sensor.
2. The system's primary states include Idle, Remote-Controlled, and Autonomous-Avoidance
3. Translates the app's commands into specific PWM signals and GPIO states for the L298N motor driver, controlling the trolley's movement.
4. Simultaneously, the system's autonomous obstacle avoidance function is active.
5. Bluetooth controls are listed on the below table:

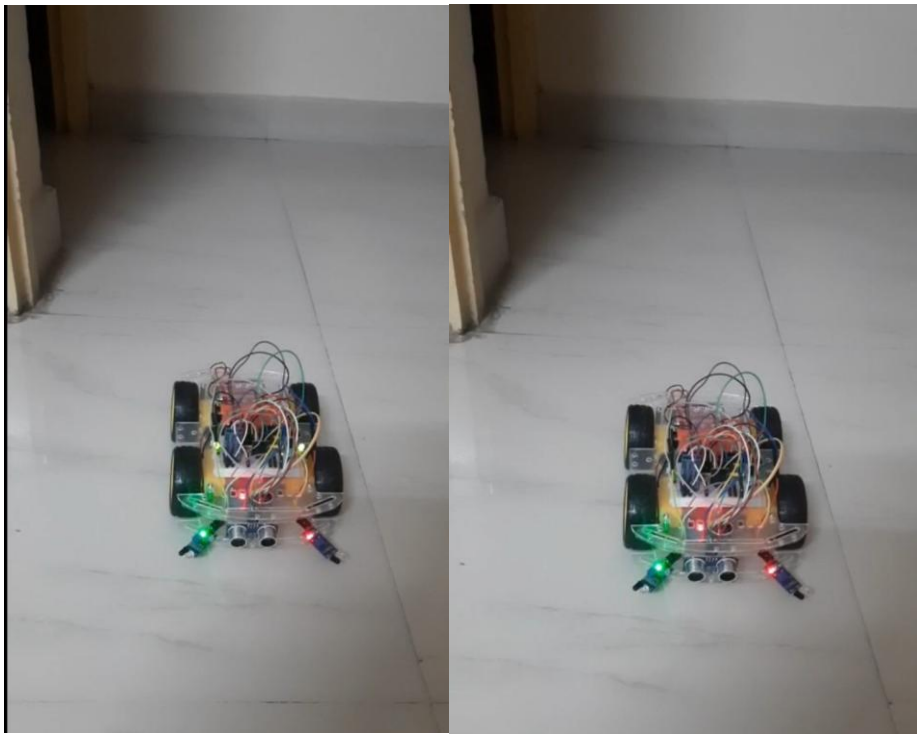
Command Key	Function / Action
'1'	Move Forward
'2'	Stop
'3'	Blink both LEDs thrice (Hazard light)
'4'	Reverse / Stop Reverse
'5'	Right Turn (90°)
'6'	Left Turn (90°)
'0'	Autonomous Mode (Obstacle Avoidance)
'h'	Play Nokia Tune
'i'	Play Happy Birthday
'j'	Play Airtel Theme (AR Rahman)
'k'	Play Merry Christmas
's'	Slow Speed (motorSpeed = 100)
'm'	Medium Speed (motorSpeed = 150)
'f'	Fast Speed (motorSpeed = 255)



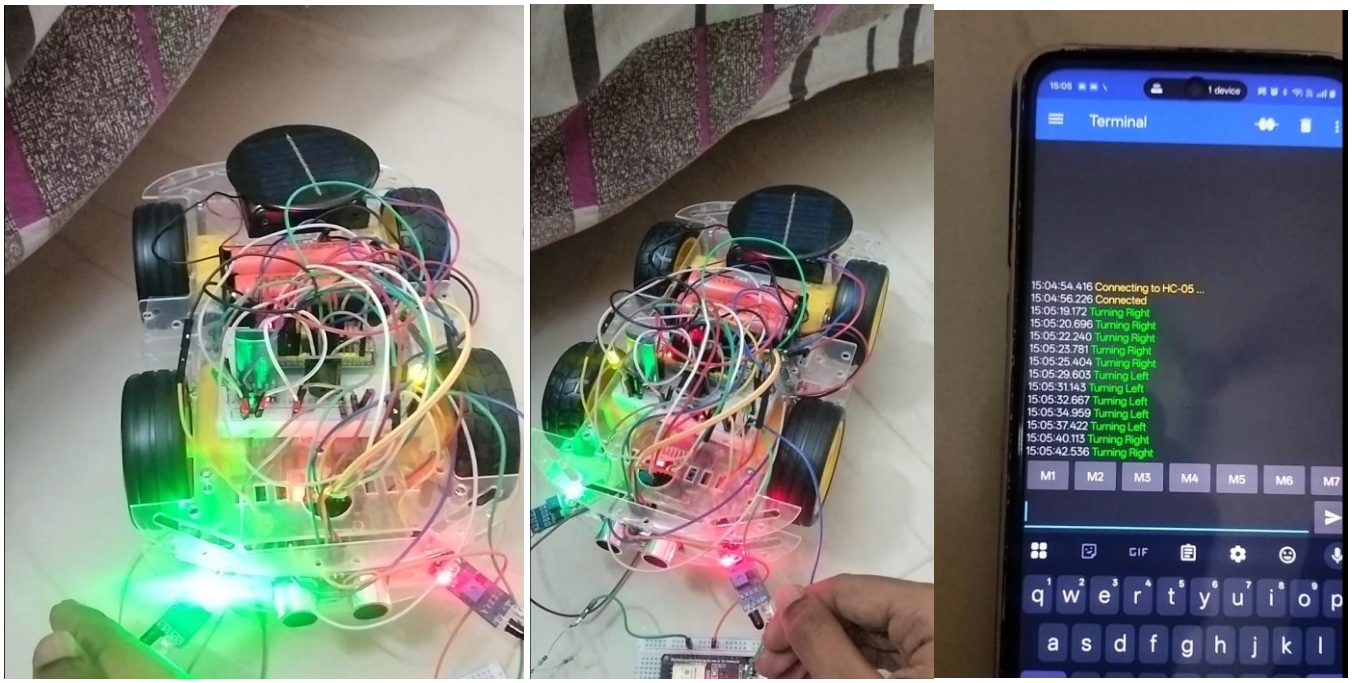
Front and back view of the car



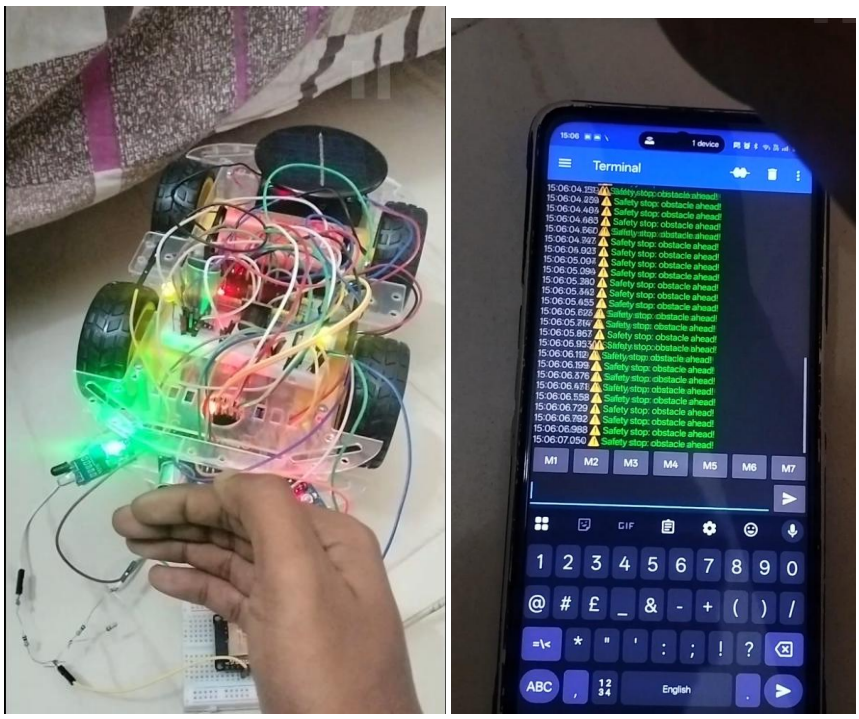
Reverse mode (buzzer plays nokia tune)



Hazard lights blinking



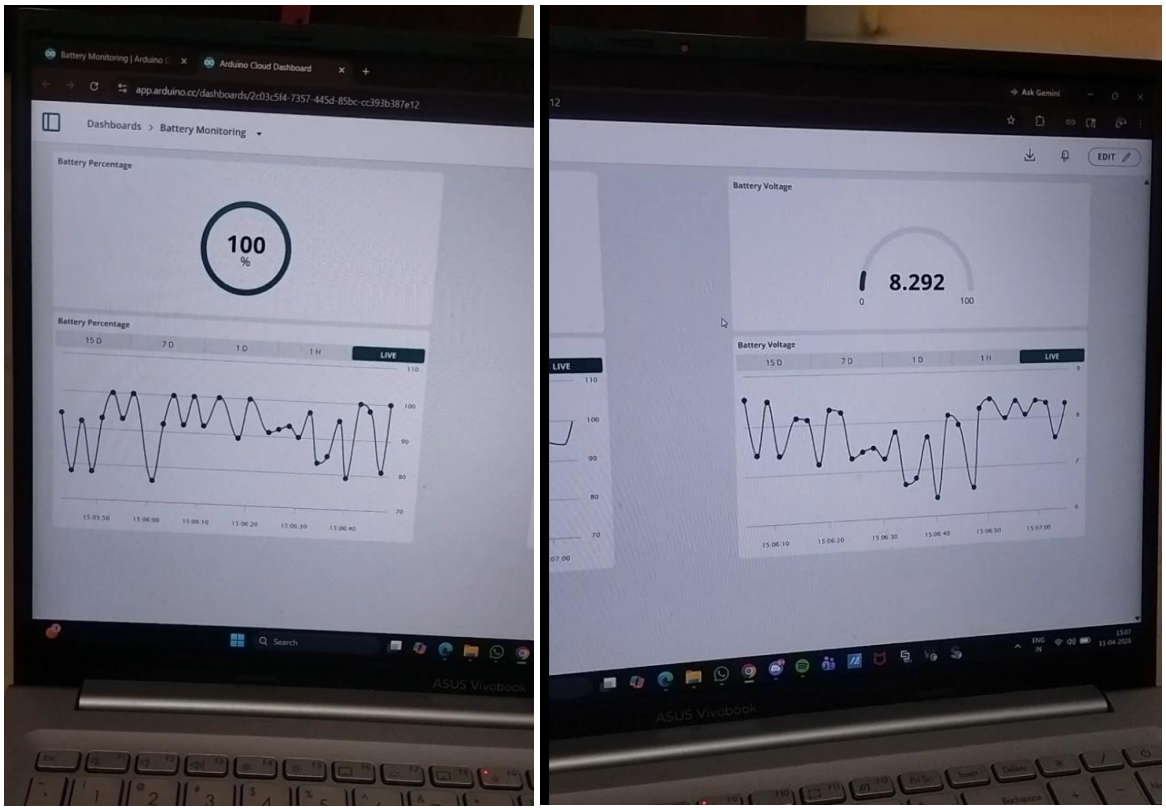
IR Sensor detects obstacle and the vehicle turns the respective direction while blinking the respective LED (left/right). The current action is also displayed on the serial bluetooth monitor



Ultrasonic sensor detects obstacle in front and stops the car. The buzzer continuously beeps and both the LEDs are turned ON. It's displayed on the bluetooth monitor

Solar cell continuously charges the battery when light is present

Buzzer used to play various tunes via PWM signals



Live battery status shown on cloud

EXECUTION PROCEDURE:

1. Hardware Setup Verification

All components (STM32, sensors, motor driver, battery, TP4056, solar panel) were connected as per the circuit diagram. Voltage levels were verified using a multimeter. Battery output and resistor connections were verified.

2. Power System Testing (Battery + Solar + TP4056)

Connected solar panel to TP4056 input. Red LED ON → Charging and Blue LED ON → Fully charged

3. Ultrasonic Sensor (HC-SR04) Testing & Calibration

Sent trigger pulse from STM32. Measured echo time using timer. Calculated distance using formula

4. IR Sensors Testing & Calibration

- Checked digital output:
    - LOW → Obstacle detected
    - HIGH → No obstacle
- Adjusted sensor sensitivity (potentiometer).
    - Left IR → detects left obstacle
    - Right IR → detects right obstacle

5. Motor Driver (L298N) Testing

Applied manual HIGH/LOW signals to IN1–IN4 pins and verified forward rotation, reverse rotation and left/right turning. Also adjusted PWM duty cycle: 100 → slow speed, 150 → medium speed, 255 → maximum speed.

6. Bluetooth Module (HC-05) Testing

Sent commands via Bluetooth terminal. '1' → Forward, '2' → Stop, '0' → Autonomous mode. Verified response via serial monitor.

7. Buzzer and LED Testing

Tested PWM-based tone generation for different melodies (Nokia, Happy Birthday).

LED - Hazard blinking, Direction indication (left/right)

8. Integration of Sensors with STM32

- Ultrasonic → stop vehicle
- IR → change direction

9. Autonomous Mode Testing

Activated using command '0'. Move forward when no obstacle, Stop if obstacle ahead, Turn based on IR sensors.

11. Final System Integration Testing

All modules operated simultaneously, Sensors + motors + Bluetooth + power system

CHALLENGES FACED:

- **Sensor Accuracy and Noise:**
  Ultrasonic and IR sensors occasionally produced inconsistent readings due to environmental factors like surface reflections and ambient light, requiring careful calibration and filtering.
- **Real-Time Processing Constraints:**
  Managing multiple inputs (sensors, Bluetooth commands) simultaneously while ensuring smooth motor control was challenging within limited microcontroller resources.
- **Motor Control Stability:**
  Achieving precise movement and avoiding jerky motion required fine-tuning of PWM signals and proper synchronization between both motors.
- **Power Management Integration:**
  Designing a stable power system that integrates battery supply with solar charging was complex, especially in ensuring consistent voltage levels.
- **TP4056 Charging Limitations:**
  Handling charging efficiency and ensuring safe operation (avoiding overcharging or overheating) required proper circuit understanding and careful implementation.
- **Solar Panel Efficiency Variations:**
  The output of the solar cell varied significantly with light conditions, making it difficult to rely on it as a consistent power source.
- **Bluetooth Communication Delays:**
  Occasional latency and signal interruptions affected real-time manual control, especially in indoor environments with interference.

- **System Integration Complexity:**
  Combining multiple modules (sensors, motor driver, Bluetooth, power system) into a single cohesive system required extensive testing and debugging.
- **Space and Wiring Constraints:**
  Managing compact hardware layout while avoiding loose connections and interference between components was challenging.
- **Debugging and Testing:**
  Identifying faults across hardware and software simultaneously required iterative testing and troubleshooting.

APPLICATIONS:

- **Smart Logistics and Material Handling:**
  The platform can be used for automated transport of goods within warehouses, factories, and campuses, reducing manual effort and improving efficiency.
- **Healthcare and Contactless Delivery:**
  It can assist in delivering medicines, food, or medical supplies in hospitals and isolation zones, minimizing human contact and enhancing safety.
- **Indoor Service Robots:**
  The system can be adapted for use in hotels, offices, and malls for tasks such as room service delivery, document transfer, or guided assistance.
- **Educational and Research Platforms:**
  Serves as a practical model for learning embedded systems, robotics, and automation concepts, making it valuable for academic and prototyping purposes.
- **Home Automation and Assistance:**
  Can be used for small-scale indoor automation tasks such as carrying items, assisting elderly individuals, or performing routine household movements.
- **Security and Surveillance Support:**
  With minor modifications (like camera integration), the platform can be used for patrolling and monitoring restricted indoor areas.
- **Energy-Efficient Autonomous Systems:**
  The integration of solar-assisted charging makes it suitable for applications requiring sustainable and low-power operation.
- **Industrial Automation:**
  Can be deployed in production environments for repetitive transportation tasks, contributing to increased productivity and reduced operational costs.

CONCLUSION:

The project successfully demonstrates the design and implementation of an embedded automated driving platform with integrated battery support for low-speed indoor navigation.

By leveraging the STM32 microcontroller, the system effectively combines sensor-based obstacle detection, real-time decision-making, and precise motor control to achieve reliable and flexible operation across manual, assisted, and autonomous modes.

A key outcome of this work is the integration of a solar-assisted charging system using a TP4056 module, which enhances energy efficiency and supports sustainable operation. This feature reduces dependence on conventional charging methods while ensuring safe and consistent battery management, thereby improving the system's overall reliability and usability.

The developed platform is compact, cost-effective, and scalable, making it suitable for a wide range of applications such as smart logistics, contactless delivery, and indoor automation.

DEMO VIDEO LINK:

https://drive.google.com/file/d/1qoGVO4EL_VkTSq7dn8tb9yL1snF4DSZ3/view?usp=sharing

ENTIRE CODE:

```
#include "main.h"
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
UART_HandleTypeDef huart1,huart3;
TIM_HandleTypeDef htim1,htim3;
#define IR1_PIN_GPIO GPIOA
#define IR1_PIN_PIN GPIO_PIN_0
#define IR2_PIN_GPIO GPIOA
#define IR2_PIN_PIN GPIO_PIN_1
#define TRIG_PIN_GPIO GPIOA
#define TRIG_PIN_PIN GPIO_PIN_2
#define ECHO_PIN_GPIO GPIOA
#define ECHO_PIN_PIN GPIO_PIN_3
#define LED1_PIN_GPIO GPIOB
#define LED1_PIN_PIN GPIO_PIN_0
#define LED2_PIN_GPIO GPIOB
#define LED2_PIN_PIN GPIO_PIN_1
#define ONBOARD_LED_GPIO GPIOC
#define ONBOARD_LED_PIN GPIO_PIN_13
#define BUZZER_GPIO GPIOA
#define BUZZER_PIN GPIO_PIN_8
#define IN1_GPIO GPIOB
#define IN1_PIN GPIO_PIN_5
#define IN2_GPIO GPIOB
#define IN2_PIN GPIO_PIN_6
#define IN3_GPIO GPIOB
#define IN3_PIN GPIO_PIN_7
#define IN4_GPIO GPIOB
#define IN4_PIN GPIO_PIN_8
#define ENA_GPIO GPIOA
```

```
#define ENA_PIN GPIO_PIN_6

#define ENB_GPIO GPIOA

#define ENB_PIN GPIO_PIN_7


int motorSpeed=150;

#define DIST_THRESHOLD 15.0f

volatile char btCommand='0',newCommand='0';

#define NOTE_C4 262

#define NOTE_D4 294

#define NOTE_E4 330

#define NOTE_F4 349

#define NOTE_G4 392

#define NOTE_A4 440

#define NOTE_AS4 466

#define NOTE_B4 494

#define NOTE_C5 523

#define NOTE_D5 587

#define NOTE_E5 659

#define NOTE_F5 698

#define NOTE_G5 784

#define NOTE_A5 880

#define NOTE_AS5 932

#define NOTE_B5 988

#define NOTE_C6 1047

#define REST 0

#define NOTE_FS5 740

#define NOTE_GS5 831

#define NOTE_CS4 277

#define NOTE_DS4 311

#define NOTE_FS4 370

#define NOTE_GS4 415

#define NOTE_CS5 554

#define NOTE_DS5 622

#define NOTE_D6 1175

#define NOTE_DS6 1245

int tempo1=140,tempo2=180,tempo3=140;

int melody1[]={NOTE_C4,4,NOTE_C4,8,NOTE_D4,-
4,NOTE_C4,-4,NOTE_F4,-4,NOTE_E4,-
2,NOTE_C4,4,NOTE_C4,8,NOTE_D4,-4,NOTE_C4,-
4,NOTE_G4,-4,NOTE_F4,-2};

int
melody2[]={NOTE_E5,8,NOTE_D5,8,NOTE_F4,4,NOT
E_G4,4,NOTE_C5,8,NOTE_B4,8,NOTE_D4,4,NOTE_E
4,4,NOTE_B4,8,NOTE_A4,8,NOTE_C4,4,NOTE_E4,4,
NOTE_A4,2};

int
melody3[]={NOTE_C5,4,NOTE_F5,4,NOTE_F5,8,NOT
E_G5,8,NOTE_F5,8,NOTE_E5,8,NOTE_D5,4,NOTE_D
5,4,NOTE_D5,4,NOTE_G5,4,NOTE_G5,8,NOTE_A5,8,
NOTE_G5,8,NOTE_F5,8,NOTE_E5,4,NOTE_C5,4,NOT
E_C5,4};

int horn2Tempo=114;

int horn2Melody[]={NOTE_D5,-4,NOTE_E5,-
4,NOTE_A4,4,NOTE_E5,-4,NOTE_FS5,-
4,NOTE_A5,16,NOTE_G5,16,NOTE_FS5,8,NOTE_D5,-
4,NOTE_E5,-
4,NOTE_A4,2,NOTE_A4,16,NOTE_A4,16,NOTE_B4,1
6,NOTE_D5,8,NOTE_D5,16,NOTE_D5,-4,NOTE_E5,-
4,NOTE_A4,4,NOTE_E5,-4,NOTE_FS5,-
4,NOTE_A5,16,NOTE_G5,16,NOTE_FS5,8,NOTE_D5,-
4,NOTE_E5,-4,NOTE_A4,2};

int horn2Notes,horn2Whole;
```

```
int horn3Tempo=120;

int
horn3Melody[]={NOTE_A4,4,NOTE_A4,8,NOTE_A4,
8,NOTE_A4,4,NOTE_E5,8,REST,8,NOTE_B4,8,NOTE_
C5,8,NOTE_B4,8,NOTE_A4,8,NOTE_G4,4,NOTE_B4,4
,NOTE_A4,8,NOTE_B4,8,NOTE_A4,8,NOTE_G4,8,NO
TE_A4,8,NOTE_B4,8,NOTE_C5,8,REST,8,NOTE_B4,2,
NOTE_A4,8,NOTE_G4,8,NOTE_E4,2,NOTE_A5,4,NOT
E_E5,4,NOTE_G5,4,NOTE_E5,8,REST,8,NOTE_E5,4,N
OTE_D5,4,NOTE_C5,4,NOTE_A4,4,NOTE_C5,4,NOTE
_D5,4,NOTE_E5,4,NOTE_F5,8,REST,8,NOTE_D5,2,N
OTE_C5,8,NOTE_B4,8,NOTE_A4,8};

int horn3Notes,horn3Whole;

int
notes1,notes2,notes3,wholenote1,wholenote2,who
lenote3,divider=0,noteDuration=0;

char uartbuf[128];

void DWT_Delay_Init(void){CoreDebug-
>DEMCR|=CoreDebug_DEMCR_TRCENA_Msk;DWT-
>CTRL|=DWT_CTRL_CYCCNTENA_Msk;}

uint32_t micros(void){return DWT-
>CYCCNT/(HAL_RCC_GetHCLKFreq()/1000000);}

void delayMicroseconds(uint32_t us){uint32_t
s=micros();while((micros()-s)<us){__NOP();}}

void tone_play(uint16_t f){if(!f)return;uint32_t
c=HAL_RCC_GetPCLK2Freq(),p=0,a=(c/f)-
1;while(a>0xFFFF){p++;a=(c/(p+1)/f)-
1;if(p>0xFFFF)break;}__HAL_TIM_SET_PRESCALER(
&htim1,p);__HAL_TIM_SET_AUTORELOAD(&htim1,
a);__HAL_TIM_SET_COMPARE(&htim1,TIM_CHANN
EL_1,a/2);HAL_TIM_PWM_Start(&htim1,TIM_CHAN
NEL_1);}

void
tone_stop(void){HAL_TIM_PWM_Stop(&htim1,TIM
_CHANNEL_1);__HAL_TIM_SET_COMPARE(&htim1,
TIM_CHANNEL_1,0);}

float measure_distance_cm(void){

HAL_GPIO_WritePin(TRIG_PIN_GPIO,TRIG_PIN_PIN,
GPIO_PIN_RESET);

delayMicroseconds(2);

HAL_GPIO_WritePin(TRIG_PIN_GPIO,TRIG_PIN_PIN,
GPIO_PIN_SET);

delayMicroseconds(10);

HAL_GPIO_WritePin(TRIG_PIN_GPIO,TRIG_PIN_PIN,
GPIO_PIN_RESET);

uint32_t s=micros(),t=s+30000;

while(HAL_GPIO_ReadPin(ECHO_PIN_GPIO,ECHO_P
IN_PIN)==GPIO_PIN_RESET){if(micros()>t)return -1;}

uint32_t t1=micros();

while(HAL_GPIO_ReadPin(ECHO_PIN_GPIO,ECHO_P
IN_PIN)==GPIO_PIN_SET){if(micros()>t)return -1;}

uint32_t t2=micros();

return((t2-t1)*0.0343f)/2.0f;

}

uint8_t ultrasonicDetected(void){float
d=measure_distance_cm();if(d<=0)return
0;return(d>0&&d<=DIST_THRESHOLD);}

void
moveForward(){__HAL_TIM_SET_COMPARE(&htim
3,TIM_CHANNEL_1,(motorSpeed*htim3.Init.Period)
/255);__HAL_TIM_SET_COMPARE(&htim3,TIM_CHA
NNEL_2,(motorSpeed*htim3.Init.Period)/255);HAL_
GPIO_WritePin(IN1_GPIO,IN1_PIN,GPIO_PIN_SET);
HAL_GPIO_WritePin(IN2_GPIO,IN2_PIN,GPIO_PIN_
RESET);HAL_GPIO_WritePin(IN3_GPIO,IN3_PIN,GPI
O_PIN_SET);HAL_GPIO_WritePin(IN4_GPIO,IN4_PIN
,GPIO_PIN_RESET);}

void
moveBackward(){__HAL_TIM_SET_COMPARE(&hti
m3,TIM_CHANNEL_1,(motorSpeed*htim3.Init.Perio
d)/255);__HAL_TIM_SET_COMPARE(&htim3,TIM_C
HANNEL_2,(motorSpeed*htim3.Init.Period)/255);H
```

```
AL_GPIO_WritePin(IN1_GPIO,IN1_PIN,GPIO_PIN_RESET);HAL_GPIO_WritePin(IN2_GPIO,IN2_PIN,GPIO_PIN_SET);HAL_GPIO_WritePin(IN3_GPIO,IN3_PIN,GPIO_PIN_RESET);HAL_GPIO_WritePin(IN4_GPIO,IN4_PIN,GPIO_PIN_SET);}

void turnRight(){__HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_1,(motorSpeed*htim3.Init.Period)/255);__HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_2,(motorSpeed*htim3.Init.Period)/255);HAL_GPIO_WritePin(IN1_GPIO,IN1_PIN,GPIO_PIN_SET);HAL_GPIO_WritePin(IN2_GPIO,IN2_PIN,GPIO_PIN_RESET);HAL_GPIO_WritePin(IN3_GPIO,IN3_PIN,GPIO_PIN_RESET);HAL_GPIO_WritePin(IN4_GPIO,IN4_PIN,GPIO_PIN_RESET);}

void turnLeft(){__HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_1,(motorSpeed*htim3.Init.Period)/255);__HAL_TIM_SET_COMPARE(&htim3,TIM_CHANNEL_2,(motorSpeed*htim3.Init.Period)/255);HAL_GPIO_WritePin(IN1_GPIO,IN1_PIN,GPIO_PIN_RESET);HAL_GPIO_WritePin(IN2_GPIO,IN2_PIN,GPIO_PIN_RESET);HAL_GPIO_WritePin(IN3_GPIO,IN3_PIN,GPIO_PIN_SET);HAL_GPIO_WritePin(IN4_GPIO,IN4_PIN,GPIO_PIN_RESET);}

void stopMotors(){HAL_GPIO_WritePin(IN1_GPIO,IN1_PIN,GPIO_PIN_RESET);HAL_GPIO_WritePin(IN2_GPIO,IN2_PIN,GPIO_PIN_RESET);HAL_GPIO_WritePin(IN3_GPIO,IN3_PIN,GPIO_PIN_RESET);HAL_GPIO_WritePin(IN4_GPIO,IN4_PIN,GPIO_PIN_RESET);}
```